

PR-Centric Voice Manager + Autonomous Agent Workflow (OpenHands + OpenRouter)

This is the operating manual for building **Agent Mesh Observatory & Governor (V0)** with a **PR-centric, voice-first manager** workflow where agents can make progress without you at a desk.

Key idea: **iOS Shortcuts is only a voice UI**. All real work happens in GitHub (state) and the agent runner (compute).

1) Components and responsibilities

Phone + iOS Shortcuts (voice UI)

- Captures dictation (AirPods / phone mic)
- Calls the **GitHub REST API** to:
 - create a PR (or create a branch + PR stub)
 - post a PR comment
 - apply/remove PR labels (state)
- **Never executes code** and does not run the agent

GitHub (control plane)

- The **PR is the single source of truth** for each feature
- PR **labels** implement a state machine (start/pause/needs-input)
- PR **comments** are the command channel (from you and from agents)
- **CI checks** are the hard quality gate before merge

GitHub Actions (compute)

- Runs OpenHands in an isolated runner
- Checks out the PR branch
- Executes agent steps, runs tests, writes code
- Commits and pushes to the PR branch
- Posts summaries/questions back to the PR

OpenHands (agent framework)

- Orchestrator agent coordinates role agents (backend/frontend/sdk/test/docs)
- Uses repo context + CI feedback to iteratively reach “ready-to-merge”

OpenRouter (LLM router)

- One API key → many models
- You pick one **default model** for consistency + a **fallback list** for reliability

- Token usage is reported per run (used for cost control)

Optional: self-hosted runner (Mac/Linux)

- Only if you need special builds (e.g., Xcode/macOS) or private deps not suitable for hosted runners
 - Not required for the baseline workflow
-

2) Modes

Voice Manager Mode (you are “product manager”)

- You create PRs and set priorities by voice
- You answer agent questions (yes/no or multiple-choice)
- You start/pause agent work by changing PR labels

Autonomous Mode (event-driven)

Autonomy is **not** a schedule. It's a loop driven by GitHub events: 1) PR labeled **agent-takeover** → agent workflow runs 2) Agent pushes commits → CI runs 3) CI completes (workflow_run) → agent workflow resumes to fix failures or continue 4) Repeat until: - **needs-input** (agent pauses and asks you) - **blocked** (agent can't proceed safely) - **ready-to-merge** (all checks pass + acceptance criteria met)

3) PR-centric contract

3.1 One PR per feature

- Small, reviewable increments (target: 1–3 hours of agent work per PR)
- Clear acceptance criteria in the PR description

3.2 PR template (required)

Include at the top of every PR description: - **Goal** - **Definition of done** - **Constraints / non-goals** - **Current status** (✅🟡❌) - **Next steps** (numbered) - **Questions for Sam** (must be yes/no or multiple-choice)

3.3 Branch & PR naming

- Branch: or
 - PR title: /
-

4) PR label state machine (required)

Create these labels and use them consistently: - **agent-takeover**: agent is allowed to work and push commits to this PR - **needs-input**: agent must stop and ask questions (only yes/no or multiple-choice) -

paused: agent must not run (manual pause) - **blocked:** agent cannot continue safely; posts summary + options - **ready-to-merge:** all acceptance criteria met; CI green

State transitions (how we switch modes): - Voice Manager → Autonomous: you add **agent-takeover** (and remove **paused** if present) - Autonomous → Voice Manager: agent adds **needs-input** (or you add **paused**) - Resume autonomy: you remove **needs-input** and re-add **agent-takeover** - Finish: agent sets **ready-to-merge** when CI is green and DoD is met

5) GitHub repo setup checklist (one-time)

1) **Repo settings** - Default branch: `main` - Require PRs for changes - Require status checks to pass before merge (CI) - Enable branch protection (no direct pushes to `main`)

2) **Templates** - Add a PR template with the required sections (Goal, DoD, Constraints, Status, Next steps, Questions)

3) **Actions** - Add two workflows (see §6): - `ci.yml` (fast lint/test) - `agent_openhands.yml` (agent runner)

4) **Secrets** - `OPENROUTER_API_KEY` - (Optional) `OPENHANDS_GITHUB_TOKEN` only if you cannot use `GITHUB_TOKEN`

6) Workflows (CI + agent)

6.1 CI workflow (`ci.yml`)

Purpose: quick feedback so the agent can self-correct. - Trigger: `pull_request` - Steps: install deps, lint, unit tests, build - Targets: keep under ~5-8 minutes where possible (cost + iteration speed)

6.2 Agent workflow (`agent_openhands.yml`)

Purpose: run OpenHands autonomously on PR events. **Triggers (event-driven autonomy):** - PR labeled `agent-takeover` - PR comment command (e.g., `/agent run`) - `workflow_run` of CI completion (so the agent reacts to failures) - Optional: `push` to PR branch (re-entrant)

Agent workflow does: 1) Checkout PR branch 2) Run OpenHands in a container 3) Provide system prompt + project instructions 4) Run tests / apply fixes 5) Commit + push to same PR branch 6) Post progress summary to the PR 7) If uncertain/stuck: set **needs-input** and ask MCQ/yes-no

7) OpenHands configuration (project policy)

7.1 Role agents (recommended)

- **Orchestrator/PM**: maintains PR status block, chooses next task
- **Backend**: API + DB + processing pipeline
- **SDK**: Python/TS SDK instrumentation
- **QA/Test**: adds tests, fixes flakes, improves CI
- **Docs**: updates README + API docs

7.2 Anti-loop rules (mandatory)

- Max retries per failing command/test: **2**
- Max wall-clock per run: **10–20 minutes** then pause
- If stuck: summarize + propose 2–3 options, then set **needs-input**
- Questions must be **yes/no** or **multiple-choice** only

7.3 Safety rules (mandatory)

- Never print or exfiltrate secrets
 - No network calls except dependency installs + OpenRouter
 - No “curl random URLs”
 - Never modify GitHub settings
-

8) OpenRouter model policy (cost + reliability)

8.1 Default model (pick one)

Choose one default for consistent behavior (example: **Qwen2.5-Coder-32B**).

8.2 Fallbacks

Set a short fallback list for outages / refusals (example: GPT-OSS-120B as fallback).

8.3 Budgets

- Set a per-run max tokens / max cost policy (enforced by the runner)
 - Track monthly spend via OpenRouter usage dashboards
-

9) Voice manager runbook (day-to-day)

Start a feature

1) Create PR stub (by voice) 2) Speak acceptance criteria into PR description (or first comment) 3) Apply **agent-takeover**

When the agent asks a question

1) You get a phone notification 2) Reply by voice (Shortcut posts PR comment) 3) Remove **needs-input**, re-apply **agent-takeover**

Pause everything

- Apply **paused** (agent workflow should exit early when this label exists)

Merge

- When label is **ready-to-merge** and CI is green, merge (your choice: squash vs merge)
-

10) What we implement next (in order)

1) Finalize labels + PR template 2) Add `ci.yml` 3) Add `agent_openhands.yml` wired to labels + comments + CI completion 4) Add OpenHands project instructions + anti-loop policy 5) Create iOS Shortcuts: - New PR stub - Reply to PR - Set PR state (labels) 6) Run a pilot PR (tiny task) and iterate on prompts + budgets

State transitions (how we switch modes): - Voice manager → autonomy: add **agent-takeover** (and ensure **paused** is not present) - Autonomy → voice manager (pause): add **paused** (agent workflow exits early) - Autonomy → voice manager (questions): agent adds **needs-input** and posts MCQ/yes-no questions - Resume autonomy: remove **needs-input** (or keep) and re-add **agent-takeover** after you answer

5) GitHub repo setup checklist (Chunk 2)

Do these once: 1) **Repo settings** → **Actions** - Default `GITHUB_TOKEN` permissions: **read-only** (we elevate per workflow job). 2) Create labels (Section 4). 3) Add PR template file: `.github/pull_request_template.md` (use Section 3.2). 4) Add CODEOWNERS (optional) to enforce review. 5) Add branch protection for `main`: - Require PR before merge - Require status checks (CI) - Require linear history (optional)

6) Agent runner in GitHub Actions (Chunk 3)

We will run OpenHands as an **event-driven** GitHub Actions workflow.

6.1 Triggers

Recommended triggers: - `pull_request` (types: labeled, synchronize) - `issue_comment` on PRs (types: created) for commands like `/agent run` - `workflow_run` (when CI finishes) so the agent can immediately react to failures

6.2 Early-exit rules (cost + safety)

The workflow should do nothing unless: - PR has label **agent-takeover** - PR does **not** have label **paused**

6.3 Secrets

Store in GitHub Actions Secrets: - `OPENROUTER_API_KEY` - (Optional) `OPENROUTER_APP_NAME` and `OPENROUTER_APP_URL` for tracking

6.4 Minimal permissions

Grant only what the job needs: - `contents: write` (to push commits) **only for trusted PRs** - `pull-requests: write` (to comment on PR) - Everything else read/none

7) OpenHands configuration (Chunk 3)

7.1 Agents we'll run

Start with a small team; expand later: - **Orchestrator (PM)**: keeps PR status block updated; assigns tasks - **Implementer**: writes code - **QA**: adds tests, fixes CI failures - **Docs**: updates README / usage

7.2 Loop-control (no infinite thrash)

Hard limits per run: - Max wall time (e.g., 10–20 minutes) - Max iterations (e.g., 6) - If the same error repeats twice: set **needs-input** and ask MCQ options

7.3 Allowed tools

Keep tool access minimal: - git (commit/push) - package manager install - unit tests / lint - no deploy credentials

8) OpenRouter model policy (Chunk 4)

To avoid chaos, we choose **one default model** and a short fallback list.

Recommended policy: - Default: **Qwen2.5-Coder-32B-Instruct** (cheap/fast) - Fallback: **GPT-OSS-120B** (stronger) when Qwen fails

Rules: - Use default for normal coding + tests - Escalate to fallback only when: - repeated compilation/test failures - large refactor across many files - tricky concurrency / schema migrations

9) Voice console (iOS Shortcuts) (Chunk 4)

We only need 3 shortcuts:

- 1) **New PR** - Dictate: feature name + acceptance criteria - Creates a draft PR (or issue + PR stub) and applies **agent-takeover**
 - 2) **Reply to PR** - Dictate answer - Posts comment to PR (optionally auto-targets the most recent PR labeled `needs-input`)
 - 3) **Set PR state** - Dictate: "pause agents", "resume agents", "mark ready-to-merge" - Applies/removes labels: `paused`, `agent-takeover`, `needs-input`
-

10) Day-to-day operating procedure (Chunk 5)

Start a feature

- Use **New PR** shortcut
- PR is created with:
 - template filled (Goal/DoD/Constraints)
 - label **agent-takeover**

Let agents run

- Agents push commits and keep the PR description's status block updated.

When agents ask questions

- They set **needs-input** and post MCQ/yes-no.
- You answer with **Reply to PR**.
- You resume by re-adding **agent-takeover** (or the automation can auto-resume after your reply).

Merge

- When label is **ready-to-merge** and CI is green, you merge.
-

11) Pilot plan (Chunk 1 → Chunk 5)

We will validate the system with 3 starter PRs: 1) Repo scaffold + CI + Docker/devcontainer 2) Minimal ingest API + event schema + Postgres table 3) Minimal UI page showing recent events

Success criteria for the pilot: - You can create PRs and answer questions **entirely by voice** - Agent completes at least one PR end-to-end without manual coding - Costs and runtime are bounded (no runaway loops)

5) GitHub repo setup checklist (do this once)

5.1 Repo settings

- Enable: Issues, Pull Requests, Actions
- Protect `main`:
 - require PRs before merge
 - require status checks (CI)
 - require 1 approval (you)
 - disallow force-push

5.2 Branch/PR templates

- Add a PR template containing the fields in section 3.2
- (Optional) Add issue templates for “Feature PR request” and “Bug PR request” that your Shortcuts can populate

5.3 Labels

Create the labels from section 4 plus optional: - **agent-ok-to-push** (alias for agent-takeover if you prefer) - **security-review** (any auth/secrets changes) - **perf** (hot-path changes)

6) OpenRouter configuration (models, fallbacks, budgets)

6.1 Default + fallback strategy

Pick ONE default model for consistency. Recommended default for cost: **Qwen2.5-Coder-32B** (via OpenRouter). Fallbacks (example): 1) Qwen2.5-Coder-32B (primary) 2) GPT-OSS-120B (fallback for harder tasks) 3) Another coding-capable model you trust (optional)

6.2 Cost guardrails

- Set a per-run token budget and a max runtime
 - If the agent hits budget, it must:
 - stop
 - label **needs-input**
 - post a summary + next options
-

7) OpenHands agent design (what “the team” is)

We run one OpenHands orchestrator that can invoke role agents. Keep the role set small at first: - **Planner/Orchestrator**: breaks PR into steps, updates PR checklist, enforces budgets - **Implementer**: writes code to meet acceptance criteria - **Tester**: adds/updates tests, fixes CI failures - **Docs** (optional): updates README, API docs, changelog

Rules to prevent loops: - Max 2 attempts per failing step before asking you - Must run tests/lint before pushing - If CI fails twice for the same reason, escalate to **needs-input**

8) GitHub Actions workflows (baseline)

You need two workflows:

8.1 CI workflow (always)

Triggers: `pull_request` Jobs: lint + unit tests + build Output: PR status checks

8.2 Agent workflow (autonomy)

Triggers: - PR labeled **agent-takeover** - PR comment command (optional), e.g., `/agent run` - `workflow_run` for CI completion (so the agent can react to failures)

Behavior: - Exit immediately if label **paused** is present - If label **needs-input** is present, exit (agent must wait for you) - Otherwise: - checkout PR branch - run OpenHands with OpenRouter key - push commits - comment summary + next steps - if blocked: label **blocked** and stop

9) iOS Shortcuts (voice UI) — minimal set

9.1 Shortcut: “New Feature PR”

- Ask (dictation): PR title, goal, definition of done
- Call GitHub API to create PR (or create issue + PR stub)
- Apply label: **agent-takeover**

9.2 Shortcut: “Reply to PR”

- Ask (dictation): PR number or choose “latest needs-input”
- Dictate answer
- Post PR comment
- Optionally remove **needs-input** and re-add **agent-takeover**

9.3 Shortcut: “Pause/Resume agents”

- Dictate PR number
 - Toggle **paused** label
-

10) Day-in-the-life runbook

Starting a new feature (on bike)

1) Use "New Feature PR" 2) Agent begins work under **agent-takeover** 3) You only respond if **needs-input** appears

When the agent asks a question

1) Notification arrives 2) Use "Reply to PR" and answer 3) Shortcut resumes autonomy by restoring **agent-takeover**

When ready to merge

- Agent labels **ready-to-merge**
 - You do a quick review on phone/laptop and merge
-

11) Safety + security defaults (do not skip)

- Use least-privilege GitHub tokens for agent workflow
 - Never expose secrets to untrusted PR contexts
 - Require CI to pass before merge
 - Keep agent runner ephemeral (hosted runners) unless you have a reason
-

12) Implementation plan (chunked)

Chunk 1 — GitHub control plane (60–90 min) - Create repo, labels, PR template, branch protections

Chunk 2 — OpenRouter + secrets (15–30 min) - Create OpenRouter key, add GitHub Actions secret(s)

Chunk 3 — CI workflow (30–60 min) - Add lint/test/build workflow and verify checks appear on PRs

Chunk 4 — Agent workflow (2–4 hours) - Add OpenHands run workflow + event triggers + pause/resume label logic

Chunk 5 — iOS Shortcuts (30–90 min) - New Feature PR + Reply to PR + Pause/Resume

Chunk 6 — Pilot run (1–2 hours) - Pick a tiny feature PR and run the full loop end-to-end

If you want, next we can implement Chunk 1 as a checklist tailored to your repo name and stack (language/framework), then move to Chunk 2–6.

5.3 Secrets

Add repo secrets: - `OPENROUTER_API_KEY` - (If needed for pushing commits beyond the default token)
`AGENT_GIT_TOKEN`

5.4 Label creation

Create labels listed in section 4 plus: - `agent-backend`, `agent-sdk`, `agent-ui`, `agent-ci`, `agent-docs` (routing only) - `priority-high` (routing only)

6) OpenHands runner (Actions) — how autonomy runs

6.1 Event triggers (autonomy is event-driven)

Configure workflows to trigger on: - PR labeled `agent-takeover` - PR comments that start with a command (e.g., `/agent`) - `workflow_run` for your CI workflow completing (so agent can react to failures)

6.2 Guardrails (prevents loops and runaway costs)

- Per-run wall clock limit: 10–20 minutes
- Max iterations per PR per day (e.g., 6)
- If the same error repeats twice: add `needs-input` and ask you
- Never merge automatically; only set `ready-to-merge`

6.3 What the agent is allowed to do

- Checkout PR branch, edit files, run tests, commit/push to PR branch
 - Post a “Run Summary” comment with:
 - what changed
 - what still fails
 - what it will do next
 - questions (MCQ/yes-no only)
-

7) OpenRouter model policy (cost + reliability)

7.1 Default model

Pick one default model and stick with it for consistent behavior. Recommended defaults for cost/perf: - `qwen2.5-coder-32b-instruct` (cheap, strong for scaffolding)

7.2 Fallbacks

Set a fallback list for reliability: - fallback #1: `gpt-oss-120b` - fallback #2: another Qwen coder variant

7.3 Budgeting

- Use a monthly OpenRouter budget cap
 - Log tokens used per agent run in the PR summary comment
-

8) iOS Shortcuts (voice UI) — minimum set

You only need 3 shortcuts to start.

8.1 “New Feature PR”

Voice prompts: - Feature name - Goal - Definition of done - Constraints/non-goals Actions: - Create a branch name + PR stub (or create issue + PR) via GitHub API - Add label: `agent-takeover` - Post a comment: “Start” + paste the PR template content

8.2 “Reply to PR (voice)”

Voice prompts: - PR number (or “latest needs-input” if you implement lookup) - Your answer Actions: - Post PR comment via GitHub API - (Optional) remove `needs-input` and add `agent-takeover`

8.3 “Set PR State”

Voice prompts: - PR number - State: agent-takeover / paused / needs-input Actions: - Apply/remove labels accordingly

9) Operating cadence (how you’ll actually work)

Morning (2–5 min)

- Ask for “Daily Digest” (optional automation) or scan notifications
- Choose 1–3 PRs to progress and set them to `agent-takeover`

During the day (manager by voice)

- When a PR hits `needs-input`, answer with the Reply shortcut
- If something looks risky, set `paused` and queue a follow-up PR

Evening (10–20 min)

- Review `ready-to-merge` PRs
 - Merge and immediately create the next PR(s)
-

10) Safety defaults (recommended)

- Default workflow token permissions read-only; elevate only as needed in the agent workflow
- Never expose secrets to untrusted PR contexts
- Require CI green + your approval before merge
- No deploy keys or cloud credentials in agent runs until V0 is stable